Best 4 features with anomlies or outliers

In [7]:
```
# === Prepare Data ===
# Step 1/5 import packages
import pandas as pd
import matplotlib.pyplot as plt
import holidays
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from xgboost import XGBRegressor
import numpy as np
################################################################################
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')   # parse with offsets
df = df.set_index('time')
df = df.dropna()

def Predicting_customers(df):
    df['year'] = df.index.year
    df['month'] = df.index.month
    df['day'] = df.index.day
    df['day_of_week'] = df.index.dayofweek

    df = df.copy()
    # yesterday's customers
    df["previous_day_customers"] = df["customers"].shift(1).fillna(0)
    df = df.copy()
        # rolling mean up to yesterday (no leakage)
    df["rolling_mean"] = df["customers"].shift(1).rolling(window=5, min_periods=
    df["rolling_mean"] = df["rolling_mean"].fillna(df["rolling_mean"].mean())
    df = df.copy()
        # exponential smoothing up to yesterday
    df["exponential_smoothing"] = df["customers"].shift(1).ewm(span=5, adjust=Fa
    df["exponential_smoothing"] = df["exponential_smoothing"].fillna(df["exponen
    df = df.copy()
        # rolling autocorrelation up to yesterday
    roll_autocorr = (
        df["customers"].shift(1)  # avoid peeking at today
        .rolling(window=14, min_periods=max(1 + 1, 3))
        .apply(lambda x: x.autocorr(lag=1), raw=False)
        )
    df["autocorrelation"] = roll_autocorr.fillna(0)  # safer default than mean
    df = df.copy()
    if not isinstance(df.index, pd.DatetimeIndex):
        raise ValueError("DataFrame index must be a DatetimeIndex.")
    fi_holidays = holidays.Finland(years=range(2019, 2025))
    holiday_idx = pd.DatetimeIndex(pd.to_datetime(list(fi_holidays.keys())))
    d = df.index.normalize()

    df["is_holiday"] = d.isin(holiday_idx).astype(int)
    df["day_before_holiday"] = (d + pd.Timedelta(days=1)).isin(holiday_idx).asty
    df["day_after_holiday"] = (d - pd.Timedelta(days=1)).isin(holiday_idx).astyp
    df = df.dropna()

    target = "customers"
    #features = [col for col in df.columns if col != target]
```

```
    features = ["previous_day_customers", "day_before_holiday", "exponential_smo
    X = df[features]

    y = df[target]

    # === Train/Test Split (time-aware, no shuffle) ===
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, shuffle=False
    )

    # === Model ===
    model = XGBRegressor(
        n_estimators=300,
        learning_rate=0.1,
        max_depth=5,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42
    )

    # === Train ===
    model.fit(X_train, y_train)

    # === Predict ===
    y_pred = model.predict(X_test)

    # === Evaluate ===
    mse  = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae  = mean_absolute_error(y_test, y_pred)
    r2   = r2_score(y_test, y_pred)

    print(f"MSE:  {mse:.2f}")
    print(f"RMSE: {rmse:.2f}")
    print(f"MAE:  {mae:.2f}")
    print(f"R²:   {r2:.2f}")

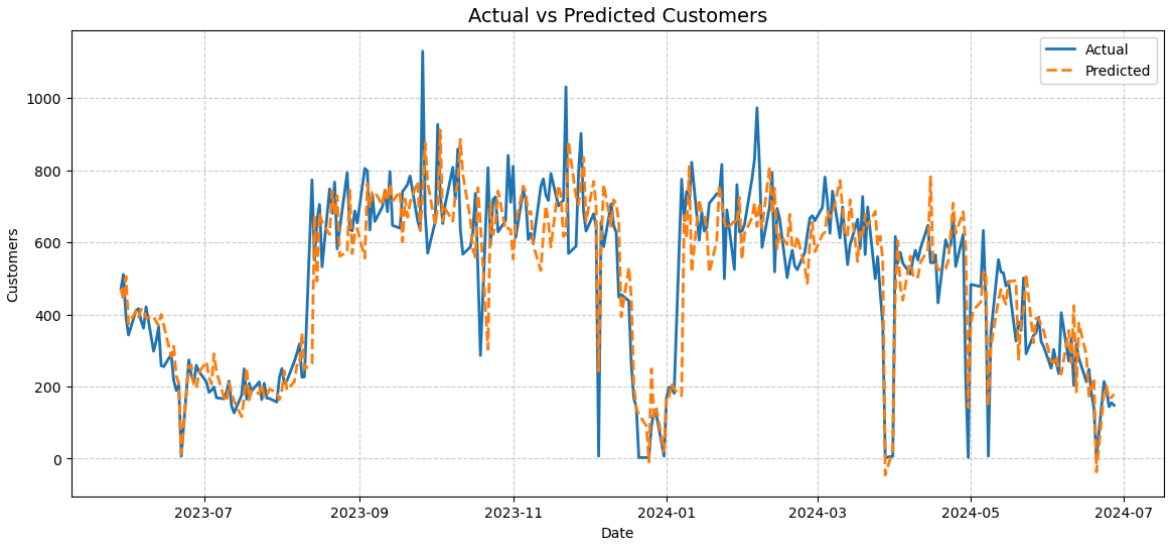
    # Align series with the test index
    results = y_test.to_frame(name="actual").copy()
    results["predicted"] = y_pred
    results.index = X_test.index  # keep the datetime index aligned

    # Plot
    plt.figure(figsize=(14, 6))
    plt.plot(results.index, results["actual"], label="Actual", linewidth=2)
    plt.plot(results.index, results["predicted"], label="Predicted", linewidth=2

    plt.title("Actual vs Predicted Customers", fontsize=14)
    plt.xlabel("Date")
    plt.ylabel("Customers")
    plt.legend()
    plt.grid(True, linestyle="--", alpha=0.6)
    plt.show()

Predicting_customers(df)
```

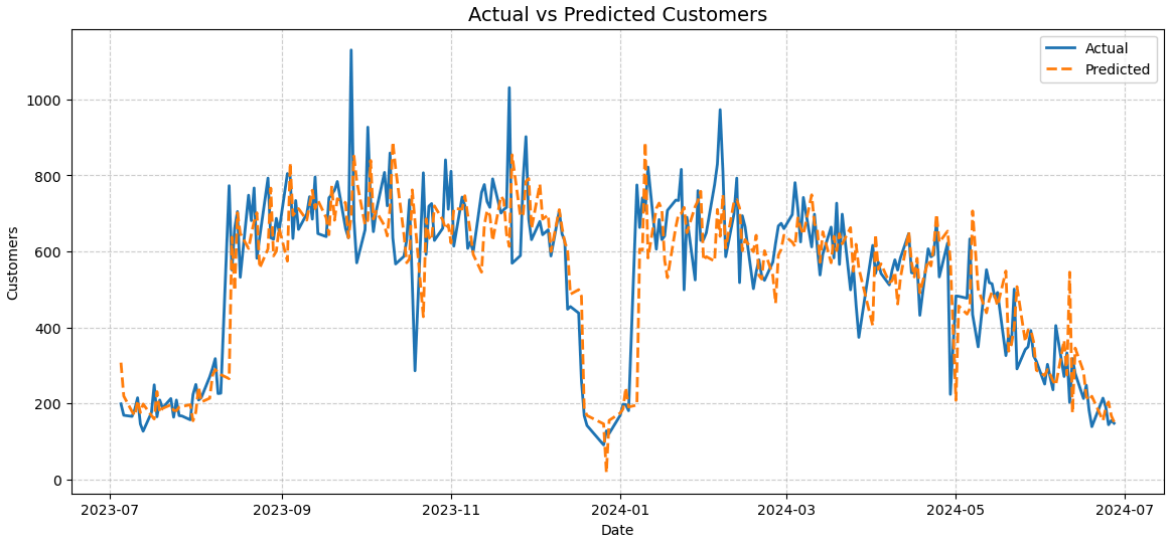
```
MSE:  15377.82
RMSE: 124.01
MAE:  88.03
R²:   0.73
```



Remove outliers or anomalies

In [8]:
```
df = df[df["customers"] > 50]
Predicting_customers(df)
```

```
MSE:  16092.32
RMSE: 126.86
MAE:  87.66
R²:   0.66
```



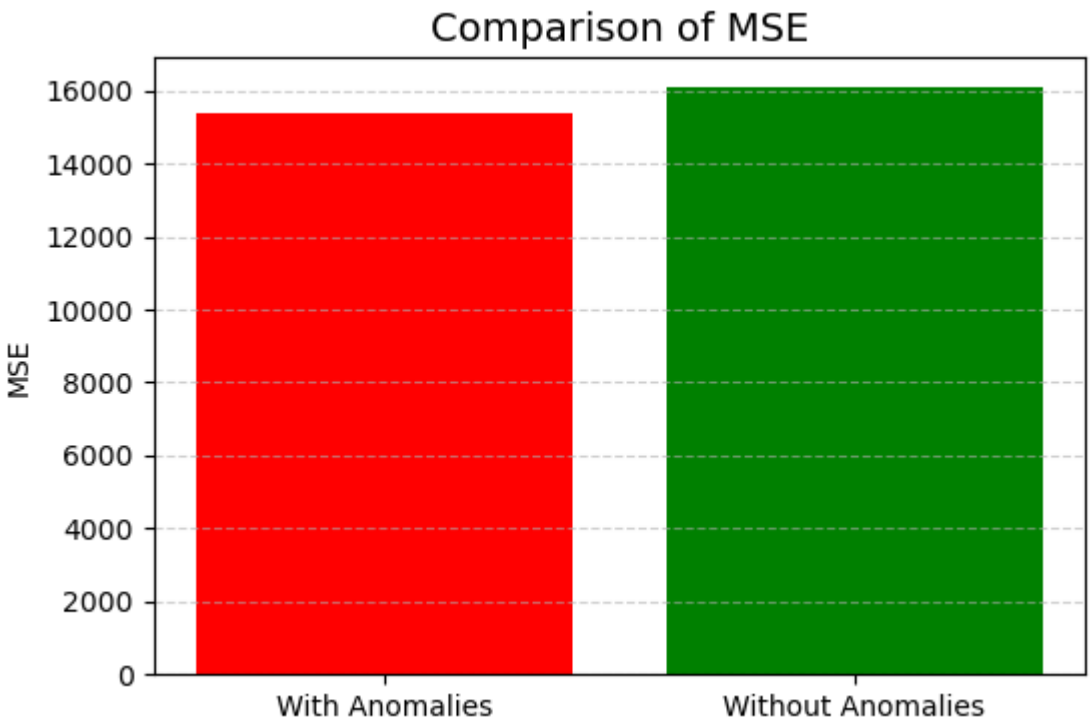
In [10]:
```
import matplotlib.pyplot as plt

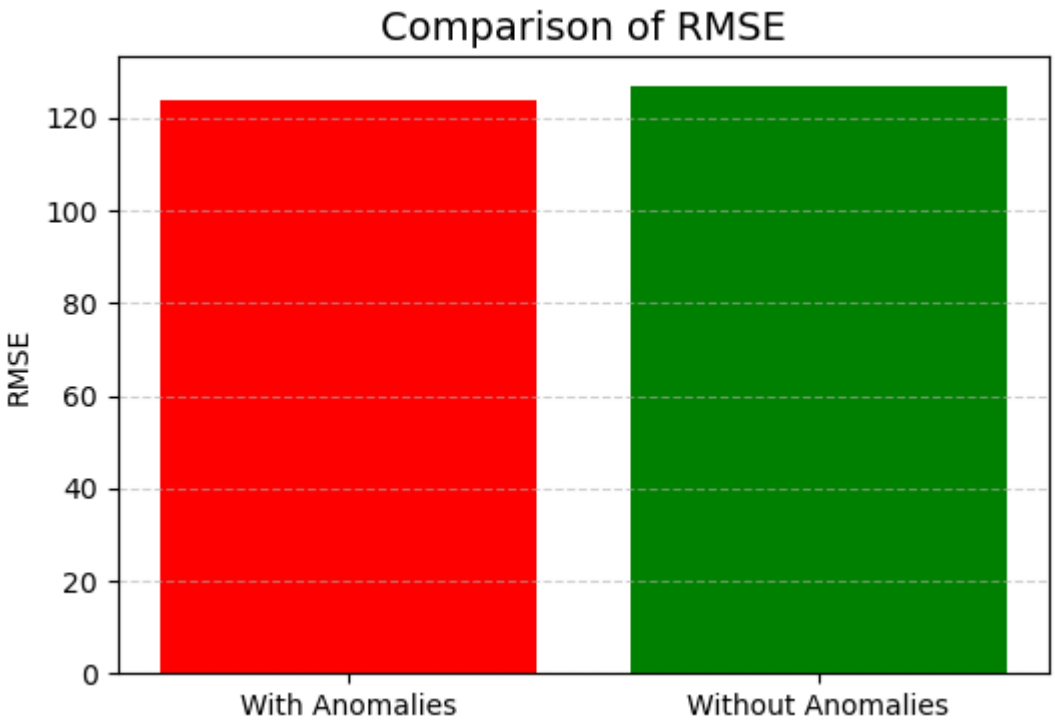
# Values from your two runs
with_anomalies     = {"MSE": 15377.82, "RMSE": 124.01, "MAE": 88.03, "R²": 0.73}
without_anomalies  = {"MSE": 16092.32, "RMSE": 126.86, "MAE": 87.66, "R²": 0.66}

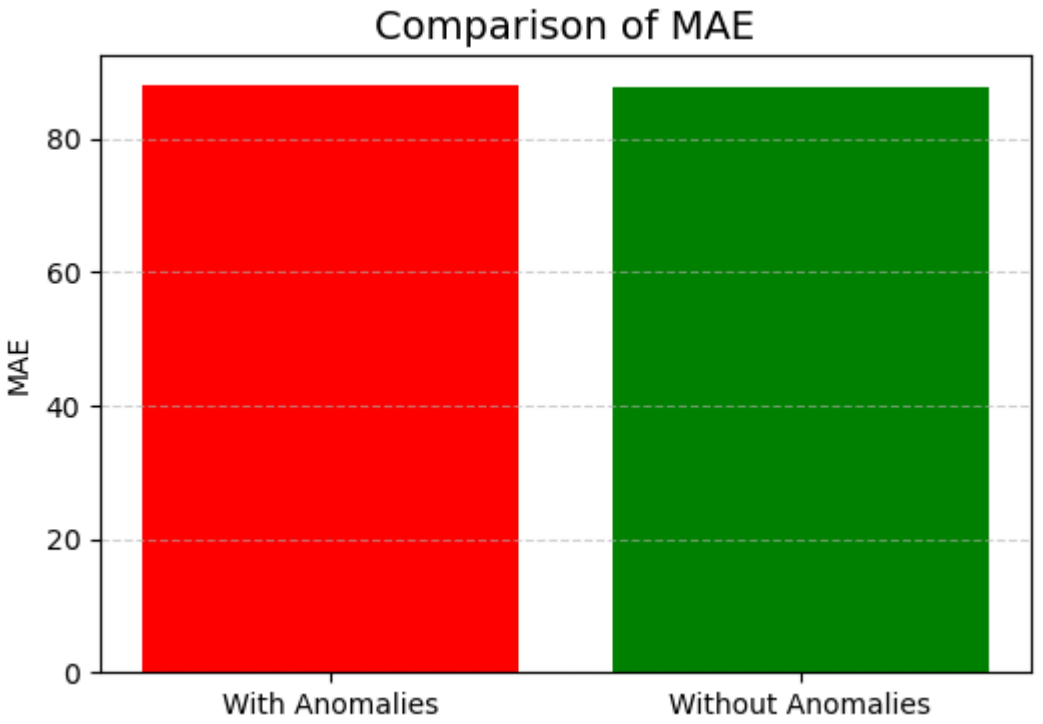
metrics = list(with_anomalies.keys())

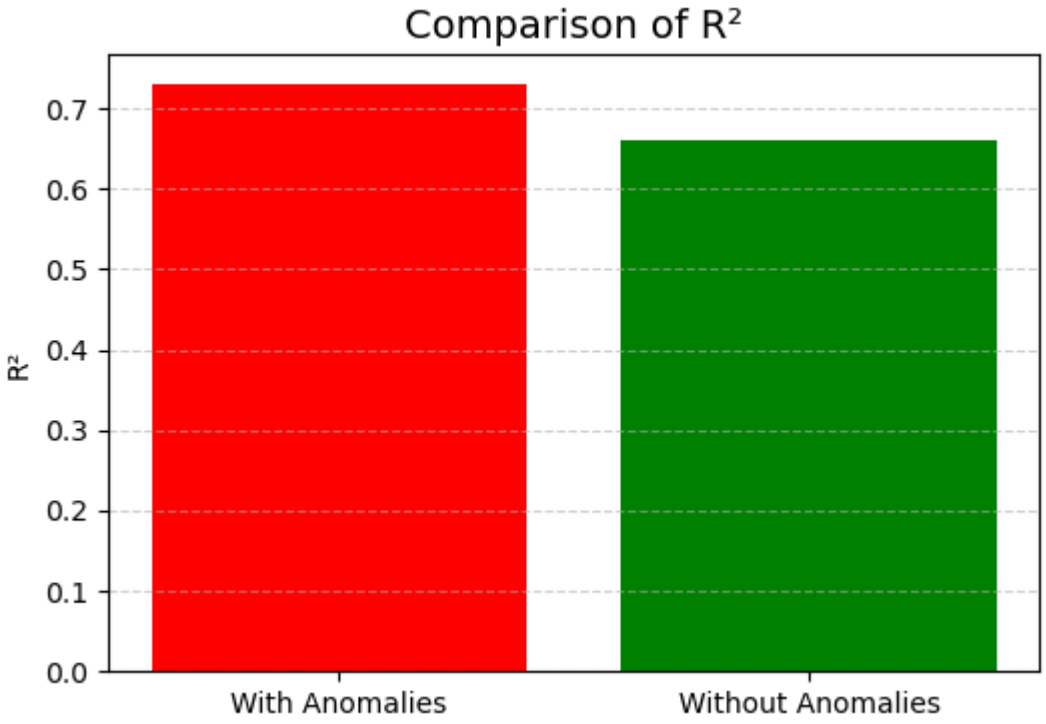
# Plot each metric separately
for metric in metrics:
```

```
    plt.figure(figsize=(6,4))
    plt.bar(["With Anomalies", "Without Anomalies"],
            [with_anomalies[metric], without_anomalies[metric]],
            color=["red", "green"])
    plt.title(f"Comparison of {metric}", fontsize=14)
    plt.ylabel(metric)
    plt.grid(axis="y", linestyle="--", alpha=0.6)
    plt.show()
```






Comparison of MAE
MAE
0
20
40
60
80
With Anomalies
Without Anomalies


Comparison of R²
R²
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
With Anomalies
Without Anomalies